JOUR 24 • MINI-PROJET

IA en enseignement supérieur

Déploiement de l'agent d'admission : Batch, Real-Time, Streaming, Edge

Kamal EL AFOUI **Version:** v1.0.0

Date : 29 octobre 2026

1. Contexte et objectifs

Une fois un modèle ou un agent entraîné, il doit être déployé pour être utilisé par les utilisateurs ou les applications. Tous les cas d'usage n'ont pas les mêmes exigences de latence, de volume de données et de connectivité : le mode de déploiement dépend donc du besoin métier.

Ce mini-projet reprend l'agent d'admission HITL du Jour 23 (scoring par règles, seuil d'escalade 70, décision finale humaine) et le déploie selon les quatre stratégies du cours. Il ajoute la couche MLOps nécessaire pour surveiller sa performance dans le temps.

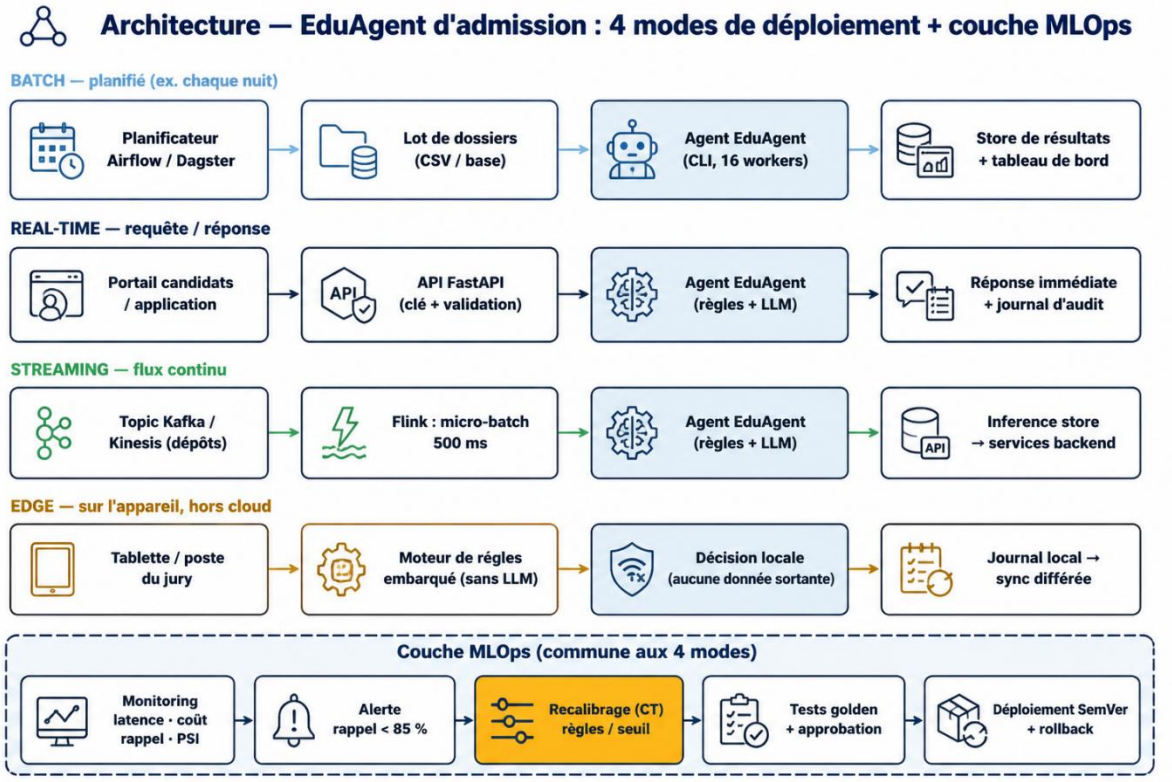
- **Principe :** mêmes règles de scoring dans les 4 modes ; seul le mode d'exécution change.
- **Simulation :** latences réseau/LLM, taux d'échec et remise batch sont simulés ; les calculs de règles sont réellement exécutés.

2. Les quatre stratégies de déploiement

Type	Quand l'utiliser ?	Outils clés	Application à EduAgent
Batch	Traitement de gros volumes à intervalles réguliers	Airflow, Dagster, Prefect	Classement nocturne de 5 000 dossiers
Real-Time	Réponse immédiate à une requête utilisateur	API REST (FastAPI)	Statut d'un candidat sur le portail
Streaming	Traitement continu d'un flux de données	Kafka, Flink, Kinesis	Pic de dépôts à la clôture des candidatures
Edge	Exécution directement sur l'appareil ou le capteur	TensorFlow Lite, Core ML, ONNX	Jury sur tablette, centre sans réseau

Edge : confidentialité maximale et latence ultra-faible sans cloud. Streaming : inférence en temps réel au cœur des flux. Batch : automatisation périodique pour les tâches de fond et le traitement de masse.

3. Architecture

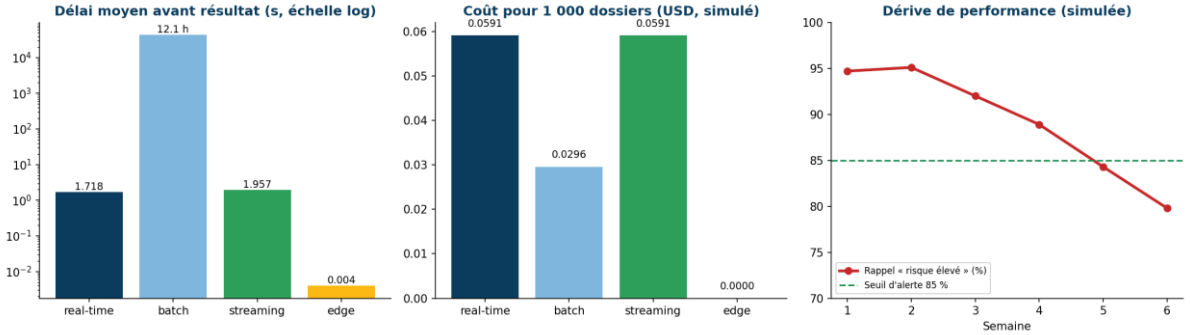


Module / endpoint	Rôle
edudeploy/scoring.py	Règles de scoring et seuil HITL (identiques au Jour 23)
edudeploy/modes.py	Simulation des 4 modes : latence, échec, repli, coût
edudeploy/drift.py	Surveillance MLOps : rappel sur décisions de la commission et PSI
edudeploy/recommend.py	Aide au choix du mode
POST /realtime/score · /edge/score	Scoring d'un dossier (clé API, validation stricte)
POST /batch/run · /stream/simulate	Lot de dossiers · simulation d'un flux
GET /recommend · /compare · /drift	Aide au choix · comparaison (admin) · dérive (admin)

4. Résultats de la simulation

Simulation de 2 000 dossiers synthétiques (graine fixe, reproductible) : python main.py compare.

Mode	Latence traitement p95	Délai avant résultat	Coût / 1 000 dossiers	Succès	Données
real-time	2287 ms	1.72 s	0.0591 $	99.95 %	vers le cloud
batch	2278.7 ms	12.1 h	0.0296 $	99.85 %	vers le cloud
streaming	2621.4 ms	1.96 s	0.0591 $	99.9 %	vers le cloud
edge	5.8 ms	4.0 ms	0.0000 $	100 %	sur l'appareil



4.1 Lecture des résultats

- **Batch :** le moins coûteux des modes cloud (0.0296 $ pour 1 000 dossiers, grâce à une remise supposée de 50 %) mais résultat disponible en moyenne après 12.1 h.
- **Real-Time :** résultat en environ 1.72 s, dépendant du LLM et du réseau ; en cas d'échec après un retry, repli en revue manuelle (risque élevé par défaut).
- **Streaming :** même coût que le temps réel, délai légèrement supérieur (1.96 s) à cause de la fenêtre de micro-batch de 500 ms, mais débit adapté aux pics.
- **Edge :** environ 4.0 ms, coût nul, aucune donnée sortante et fonctionnement hors ligne ; en contrepartie, règles seules (pas d'analyse LLM) et mise à jour des règles par distribution contrôlée.

4.2 Hypothèses et limites

Latence LLM 900–2 200 ms, aller-retour réseau 60–140 ms, taux d'échec cloud 3 %, remise batch 50 % (hypothèse à vérifier selon le fournisseur). Le coût par dossier (5.91e-5 $) est dérivé des mesures du Jour 23 (2 397 tokens et 0,000828 $ pour 14 dossiers). Ces chiffres illustrent les compromis ; ils ne remplacent pas une mesure en production.

Une panne partielle du fournisseur est testée dans le notebook (30 % d'échecs) : le mode Edge reste à 100 % de succès, les modes cloud basculent des dossiers en revue manuelle.

5. Choisir le mode de déploiement

Règle appliquée par GET /recommend, dans cet ordre : (1) pas de connectivité ou confidentialité maximale → Edge ; (2) latence tolérée d'au moins une heure → Batch ; (3) flux continu et au moins 100 000 événements par jour → Streaming ; (4) sinon Real-Time.

Scénario	Mode	Justification
Consultation du statut par les candidats sur le portail	real-time	réponse immédiate à une requête utilisateur
Classement nocturne de 5 000 dossiers	batch	résultat tolérable à l'heure : traitement planifié de gros volumes
Pic de dépôts à la clôture des candidatures	streaming	flux continu et fort volume : traitement dès l'arrivée des événements
Jury sur tablette, centre d'examen sans réseau	edge	pas de réseau fiable ou confidentialité maximale : exécution sur l'appareil
Dossiers confidentiels (handicap, dérogations) traités localement	edge	pas de réseau fiable ou confidentialité maximale : exécution sur l'appareil

Observation de monitoring (Pulse Layer, Fusion AI Performance, 28/10/2026) : le flow j23_AgentsCards a pris 54,30 s pour 21 082 tokens (11 157 en entrée, 9 925 en sortie), contre 8,05 s pour j19_AgentsMultim… et 0,04 s pour j1_copy. Une telle latence est incompatible avec une réponse interactive : ce flow se déploie en Batch ou en tâche asynchrone, pas en Real-Time.

6. MLOps versus DevOps

DevOps garantit que l'application fonctionne. MLOps garantit que l'application et le modèle IA restent performants dans le temps.

DevOps	MLOps
Gère le code	Gère code + données + modèles
CI/CD	CI/CD/CT (Continuous Training)
Tests logiciels	Validation des modèles
Déploiement application	Déploiement modèle IA
Monitoring système	Monitoring système + performance IA
Version du code	Version du code + données + modèles

6.1 Exemple : la performance baisse alors que le logiciel fonctionne

Comme la banque du cours dont la précision passe de 95 % à 75 %, l'agent voit une part croissante de dossiers à risque suivre un schéma que les règles ne détectent pas. La commission d'admission, qui relit un échantillon des dossiers, fournit la vérité terrain (boucle HITL).

Semaine	Part de cas invisibles aux règles	Rappel « risque élevé »	PSI des scores	Alerte
1	5 %	94.7 %	0.0036	—
2	5 %	95.1 %	0.0331	—
3	8 %	92 %	0.0085	—
4	12 %	88.9 %	0.0075	—
5	18 %	84.3 %	0.0296	rappel < 85 %
6	25 %	79.8 %	0.0307	rappel < 85 %

Le rappel passe sous le seuil de 85 % à la semaine 5 (84.3 %). Le PSI des scores reste au maximum à 0.0331, sous le seuil de 0,2 : la dérive n'est pas visible en surveillant seulement les distributions de données. Il faut mesurer la performance IA sur les décisions humaines. Ce jeu de production est simulé.

Étape MLOps	Action	Critère
Détection	python main.py drift · GET /drift	rappel < 85 % ou PSI > 0,2
Contention	Augmenter l'échantillon relu ; Kill Switch si grave	décisions humaines prioritaires
Remédiation	Analyser les cas manqués ; proposer règle ou seuil	cas ajoutés aux cas golden
Validation	Tests golden + approbation de 2 rôles (Jour 23)	0 écart de catégorie
Rétablissement	Déploiement SemVer minor, surveillance renforcée, rollback possible	rappel >= 85 % deux semaines de suite

7. Mise en production et sécurité

- **Conteneurisation :** Dockerfile (utilisateur non-root, healthcheck) et docker-compose.yml : service api (real-time) et job batch (profil batch) ; hébergement possible sur Azure Container Apps, Render ou Railway.
- **Secrets :** AGENT_API_KEY, ADMIN_API_KEY et OPENROUTER_API_KEY en variables d'environnement ; .env exclu de Git, .env.example fourni.
- **Moindre privilège :** clé agent distincte de la clé admin ; validation stricte des champs (pydantic) ; aucune donnée de contact transmise au modèle.
- **Edge :** sur l'appareil, les données ne quittent pas le poste ; le journal local est synchronisé plus tard ; les mises à jour de règles suivent le versionnement SemVer avec rollback.

8. Livrables et validation

Livrable	Fichier	Statut
Workflow JSON	workflow/workflow_deploiement_modes.json	Validé (12 arêtes cohérentes)
Code de l'agent	main.py + edudeploy/	12 tests réussis
Notebook	Jour24_EduAgent_Deploiement.ipynb	Exécuté sans erreur
Architecture	architecture.png	Validée
Dashboard HTML	dashboard_modes.html	Testé (navigateur)
Documentation et présentation	ce document, présentation PPTX	Validés
Déploiement	Dockerfile, docker-compose.yml, .env.example	Fournis

Commandes de reproduction

```
pip install -r requirements.txt | pytest -q | python main.py compare | python simulate.py | uvicorn main:app --port 8000 | docker compose up --build
```